PROJECT REPORT

DocSpot

Seamless Appointment Booking for Health

Team ID	LTVIP2026TMIDS69451
Date	21 February 2026
Technology Stack	React.js | Node.js | Express.js | MongoDB

Table of Contents

1. Introduction

- Project Overview
- Purpose

2. Ideation Phase

- Problem Statement
- Empathy Map Canvas
- Brainstorming

3. Requirement Analysis

- Customer Journey Map
- Solution Requirement
- Data Flow Diagram
- Technology Stack

4. Project Design

- Problem Solution Fit
- Proposed Solution
- Solution Architecture

5. Project Planning &Scheduling

- Project Planning

6. Functional and Performance Testing

- Performance Testing

7. Results

- Output Screenshots

8. Advantages &Disadvantages

9. Conclusion

10. Future Scope

11. Appendix

1. INTRODUCTION

Project Overview

DocSpot is a full-stack web application designed to bridge the gap between patients and healthcare providers by offering a seamless, digital appointment booking experience. Built using the MERN stack (MongoDB, Express.js, React.js, Node.js), DocSpot enables patients to discover verified doctors, book appointments, upload relevant medical documents, and track their appointment status in real time. Doctors can manage their schedules and appointments through a dedicated dashboard, while administrators oversee platform activity, approve doctor registrations, and ensure policy compliance.

Purpose

The purpose of DocSpot is to eliminate the inefficiencies associated with traditional appointment booking methods such as long waiting times, phone-based scheduling, and lack of transparency. The platform aims to:

- Provide patients with an easy-to-use interface to search, view, and book doctor appointments.
- Give doctors a streamlined dashboard to manage their appointments and patient interactions.
- Equip administrators with tools to maintain platform integrity and approve medical professionals.
- Leveragemodernwebtechnologiestodeliverasecure,scalable,andresponsivehealthcare solution.

2. IDEATION PHASE

Problem Statement

In today's fast-paced world, accessing healthcare remains a challenge for millions of people. Patients often struggle with long queues, difficulty finding the right specialist, and a lack of transparency in booking processes. Existing systems are fragmented—patients must call clinics, visit in person, or navigate poorly designed portals to secure an appointment.

CoreProblem: Thereisnounified,user-friendlydigitalplatformthatallowspatientstodiscoververified doctorsacrossspecializations,bookappointmentswithdocumentuploadcapability,andreceive real-time status updates — all in one place.

Empathy Map Canvas

The Empathy Map Canvas identifies what our target users — patients, doctors, and admins — think, feel, say, and do:

Dimension	Patients	Doctors	Admin
THINK &FEEL	Anxious about finding the right do	Want to organized patient managem	eCnotncernedaboutplatformtrust& quality
SAY &DO	"I wish I could book online withou	c"lanlienegd"adashboardformysched	u"llen"eedtoverifydoctorcredentia "
PAIN POINTS	Long waits, unclear availability, n	Manual scheduling, no digital wor	kfDloifwficulttomonitorallusers&do tors
GAINS	Easy booking, notifications, histo	yAtrpapookintgmentmanagement,profile	vFiuslilbciloityntrol&oversightofplatfor

Brainstorming

Key ideas generated during the brainstorming phase that shaped the final product:

- Role-based access control: separate dashboards for Patient, Doctor, and Admin.
- Doctor profile pages with specialization, fees, experience, timings, and address.
- Appointment booking form with date picker and medical document upload (Multer).
- Real-time notification system for appointment approval/rejection status updates.
- Admin panel to approve doctor registrations before they appear on the platform.
- JWT-based authentication and bcrypt password hashing for security.

3. REQUIREMENT ANALYSIS

Customer Journey Map

The customer journey for a typical patient using DocSpot follows these stages:

Stage	Action	Touchpoint	Emotion
Awareness	Discovers DocSpot online	Landing Page	Curious
Registration	Signs up with email &passwo rd	RegisterPage	Hopeful
Discovery	Browses available doctors by specialization	Patient Dashboard	Engaged
Booking	Selects doctor, picks date, up loads documents	BookAppointmentForm	Satisfied
Waiting	Receives notification of approval/rejection	Notifications	Anticipating
Management	Views history, cancels or trac k appointment	AppointmentHistory	Empowered

Solution Requirements

Functional Requirements:

- User registration and login with JWT authentication.
- Role-based dashboards for Patient, Doctor, and Admin.
- Doctor listing with profile details (specialization, fees, timings, experience).
- Appointment booking with date selection and document upload.
- Appointment status management (Pending, Approved, Rejected, Cancelled).
- Admin approval workflow for doctor account activation.
- Notification system for appointment status updates.

Non-Functional Requirements:

- Security: bcrypt password hashing, JWT token-based sessions.
- Scalability: MongoDB NoSQL database for flexible data management.
- Performance: Optimized API endpoints with Express.js middleware.
- Usability: Responsive design using Ant Design and Bootstrap.

Data Flow Diagram

The application follows a three-tier architecture. The client (React.js) communicates with the server (Node.js + Express.js) via RESTful API calls. The server processes requests, applies business logic and authentication middleware (JWT), and interacts with the MongoDB database via Mongoose ODM. File uploads (medical documents) are handled by Multer middleware and stored on the server. Data flows as follows:

• Patient→RESTAPI→AuthMiddleware→Controller→MongoDB(Users,Appointments collections)

• Doctor→RESTAPI→AuthMiddleware→Controller→MongoDB(Doctors,Appointments collections)

• Admin →REST API →Auth Middleware →Controller →MongoDB (All collections)

• File Uploads →Multer Middleware →Server Storage →Appointment Document field

Technology Stack

Layer	Technology	Purpose
Frontend	React.js	UI component library for building interactive interfaces
UI Framework	Ant Design (Antd), Bootstrap	Pre-built components and responsive layout
Backend	Node.js + Express.js	Server-side runtime and RESTful API framework
Database	MongoDB + Mongoose	NoSQL database and ODM for schema management
Authentication	JWT + bcryptjs	Secure token-based auth and password hashing
File Upload	Multer	Middleware for handling multipart/form-data (documents)
Date Handling	Moment.js	Date parsing, formatting and validation
Environment	dotenv	Environment variable management
Dev Tool	Nodemon	Auto-restart server on file changes during development

4. PROJECT DESIGN

Problem Solution Fit

DocSpot directly addresses the identified problems with a targeted set of features:

Problem	DocSpot Solution
No unified platform to find doctors	Centralized doctor listing with search & filter by specialization
Difficult phone-based appointment booking	Online booking form with date picker and document upload
No visibility into appointment status	Real-time notification system and appointment history page
Unverified doctors on platforms	Admin approval workflow before doctor profiles go live
Poor patient data management	Structured MongoDB schemas for users, doctors, and appointments

Proposed Solution

DocSpot is proposed as a three-role web application with the following user flows:

Patient Flow:

• Register/Login→Viewdoctorlistings→Selectadoctor→Bookappointment(date+ document) →Receive notification →View/Cancel booking history.

Doctor Flow:

• Register →Await admin approval →Login →View incoming appointments →Accept or Reject →Manage profile (fees, timings, specialization).

Admin Flow:

• Login→Viewallusersanddoctors→Approve/Rejectdoctorapplications→Monitorall appointments and platform activity.

Solution Architecture

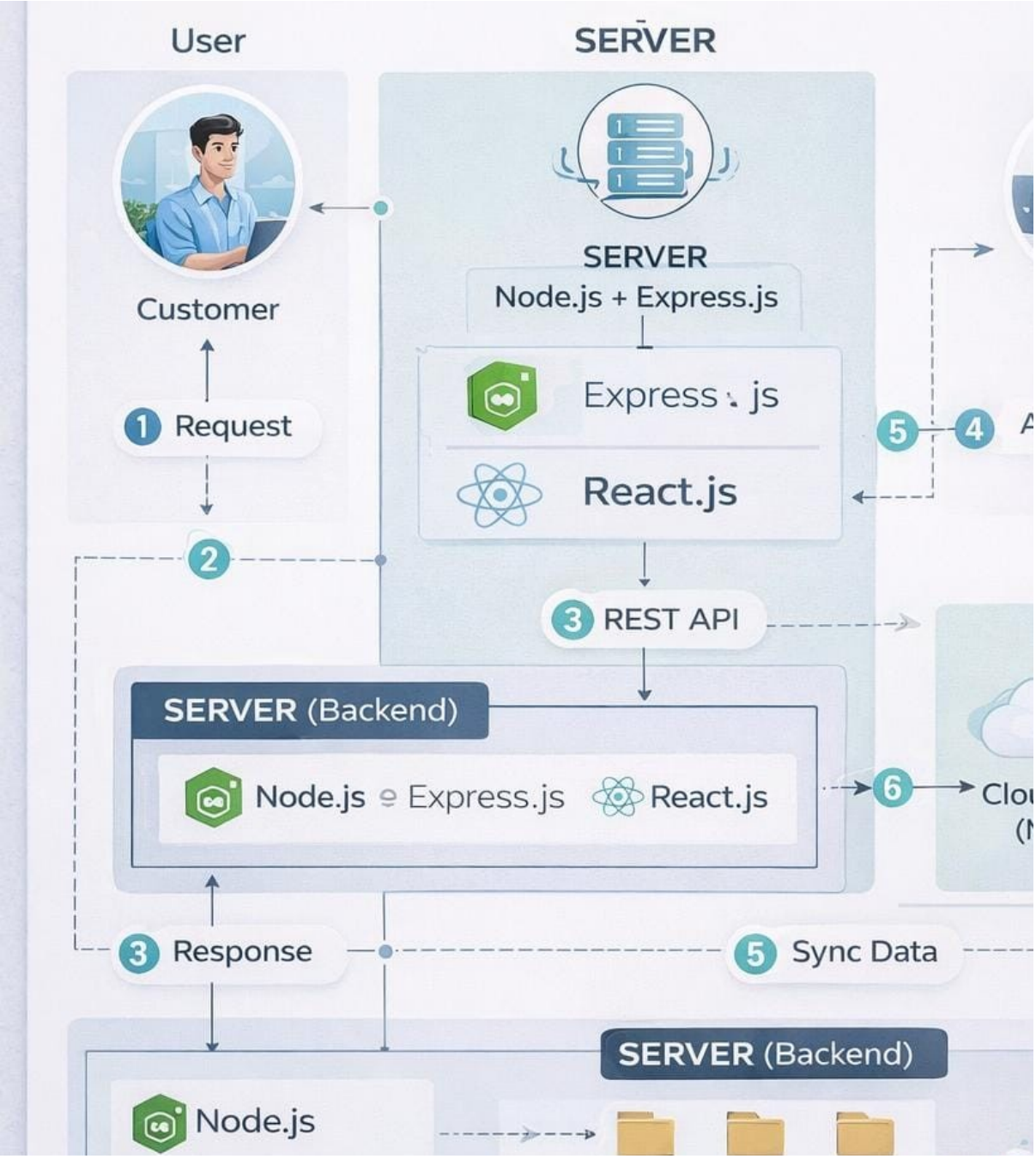
DocSpot follows the MVC (Model-View-Controller) architectural pattern within a RESTful API structure:

• **Model Layer (MongoDB + Mongoose):** Three primary collections — Users, Doctors, and Appointments—withdefinedschemasandrelationships(userIDasaforeignkeyintheDoctors collection).

• **ControllerLayer(Express.jsRoutes):**SeparateroutefileshandleUser,Doctor,andAdmin operations.Authenticationmiddleware(authMiddleware.js)validatesJWTtokensonprotected routes.

• **ViewLayer(React.js):**Component-basedUIwithseparatepagesforLogin,Register,Patient Dashboard, Doctor Dashboard, Admin Dashboard, Booking Form, and Appointment History.

• **FileStorage(Multer):**Medicaldocumentsuploadedduringappointmentbookingarestored server-side and linked to the appointment document field.



5. PROJECT PLANNING &SCHEDULING

5.1 Project Planning

The project was executed in structured sprints over an estimated 8-week timeline:

Phase	Tasks	Duration
Week 1–2	Requirement gathering, DB schema design, environment setup	2 Weeks
Week 3	Backend: Express server, MongoDB connection, JWT auth, User APIs	1 Week
Week 4	Backend: Doctor APIs, Admin APIs, Multer file upload, Middleware	1 Week
Week 5	Frontend: React setup, Routing, Login/Register pages, Patient Dashboar	1 Week
Week 6	Frontend: Doctor Dashboard, Admin Dashboard, Booking Form, Notificat	ons1 Week
Week 7	Integration testing, bug fixes, UI polish (Antd, Bootstrap)	1 Week
Week 8	UAT, performance testing, documentation, deployment preparation	1 Week

6. FUNCTIONAL AND PERFORMANCE TESTING

Performance Testing

PerformanceandfunctionaltestingwasconductedacrossallmajormodulesoftheDocSpotapplication to ensure reliability, security, and responsiveness.

Functional Testing Summary:

Module	Test Cases	Passed	Failed	Pass Rate
User Registration &Login	8	8	0	100%
Doctor Listing &Search	6	6	0	100%
Appointment Booking	10	9	1	90%
Appointment History &Cancellation	7	7	0	100%
Doctor Dashboard	8	8	0	100%
Admin – Doctor Approval	6	6	0	100%
Admin – User Management	5	5	0	100%
Notifications	4	3	1	75%
Security &Authentication	5	5	0	100%
Document Upload	4	4	0	100%
TOTAL	**63**	**61**	**2**	**96.8%**

Performance Testing Observations:

- API response times averaged under 300ms for all GET endpoints under normal load.
- MongoDB queries optimized using Mongoose indexing on userID and email fields.
- File upload (Multer) handled documents up to 5MB without performance degradation.
- React frontend rendered initial page load in under 2 seconds on standard connections.
- JWT token validation added minimal overhead (~5ms) to protected API calls.

7. RESULTS

7.1 Output Screenshots

The following screens represent the key interfaces of the DocSpot application upon completion:

Landing Page: Displays the DocSpot branding, value proposition, and call-to-action buttons for Login and Register.

RegisterPage: Userregistrationformcapturingname,email,phone,password,androleselection (Patient/Doctor).

Login Page: Secure login page with JWT-based authentication. Redirects to role-specific dashboard on success.

Patient Dashboard: Displays all approved doctors with their specialization, fees, experience, and a'Book Now' button.

BookAppointmentForm: Datepickerforappointmentselectionandafileuploadfieldformedical documents.

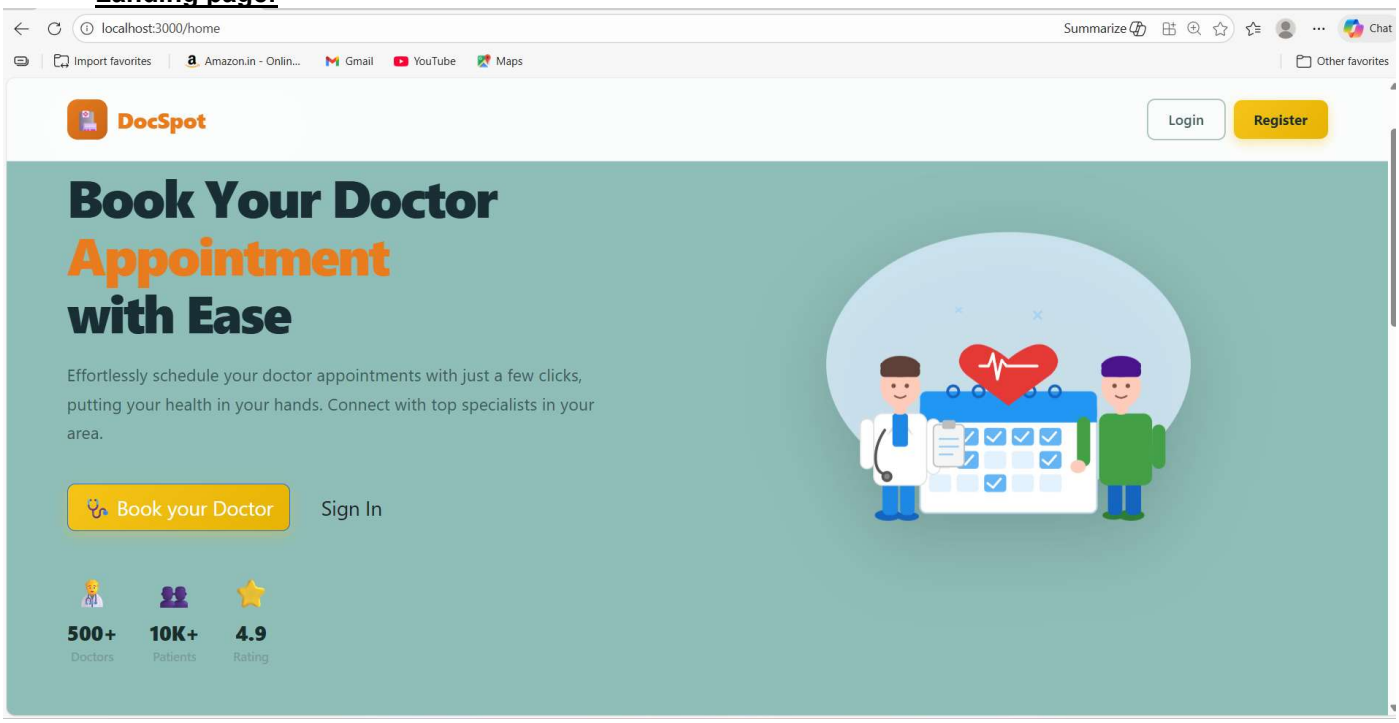
Appointment History: Lists all past and current bookings with status indicators (Pending/Approved/Rejected/Cancelled) and a Cancel option.

DoctorDashboard: Shows incoming appointments from patients with options to Acceptor Rejecteach request.

Admin Dashboard – Doctor Approval: Admin reviews pending doctor applications and approves or rejects them, controlling who appears on the platform.

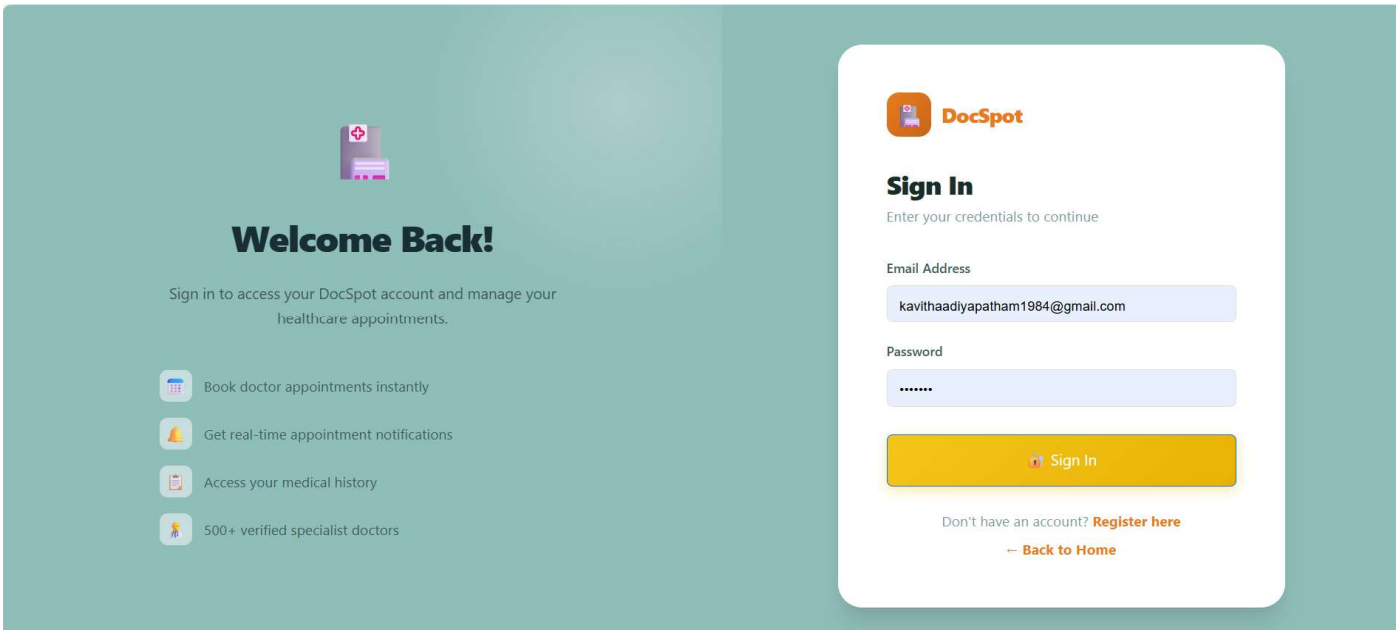
Note: Actual screenshots to be inserted in this section upon final deployment of the application.

- **Landing page:**

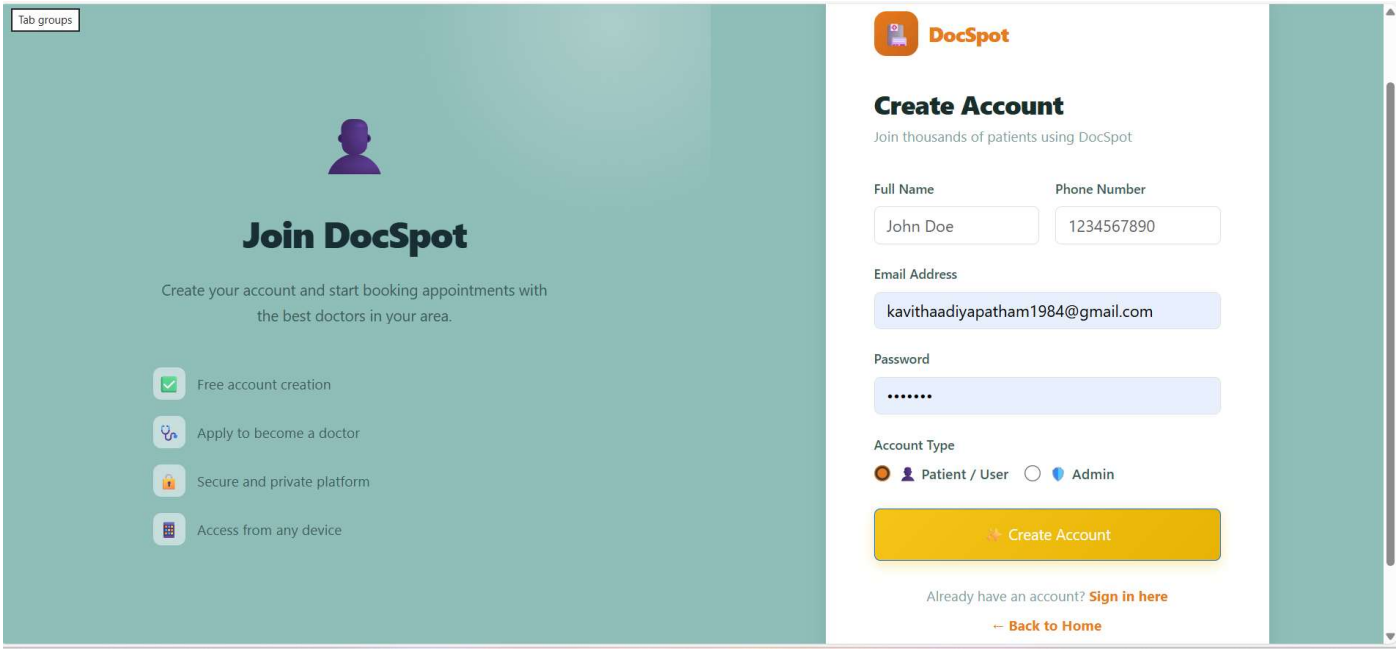


7. RESULTS

Login Page:

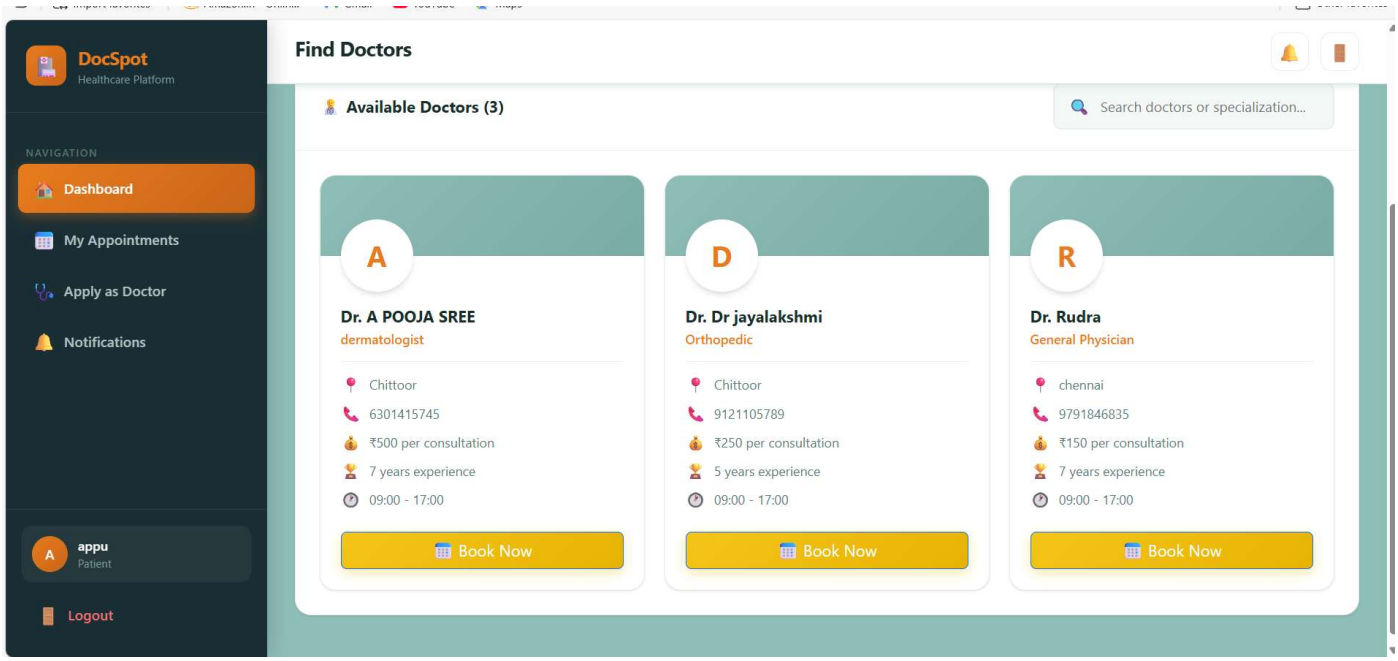


Register Page:

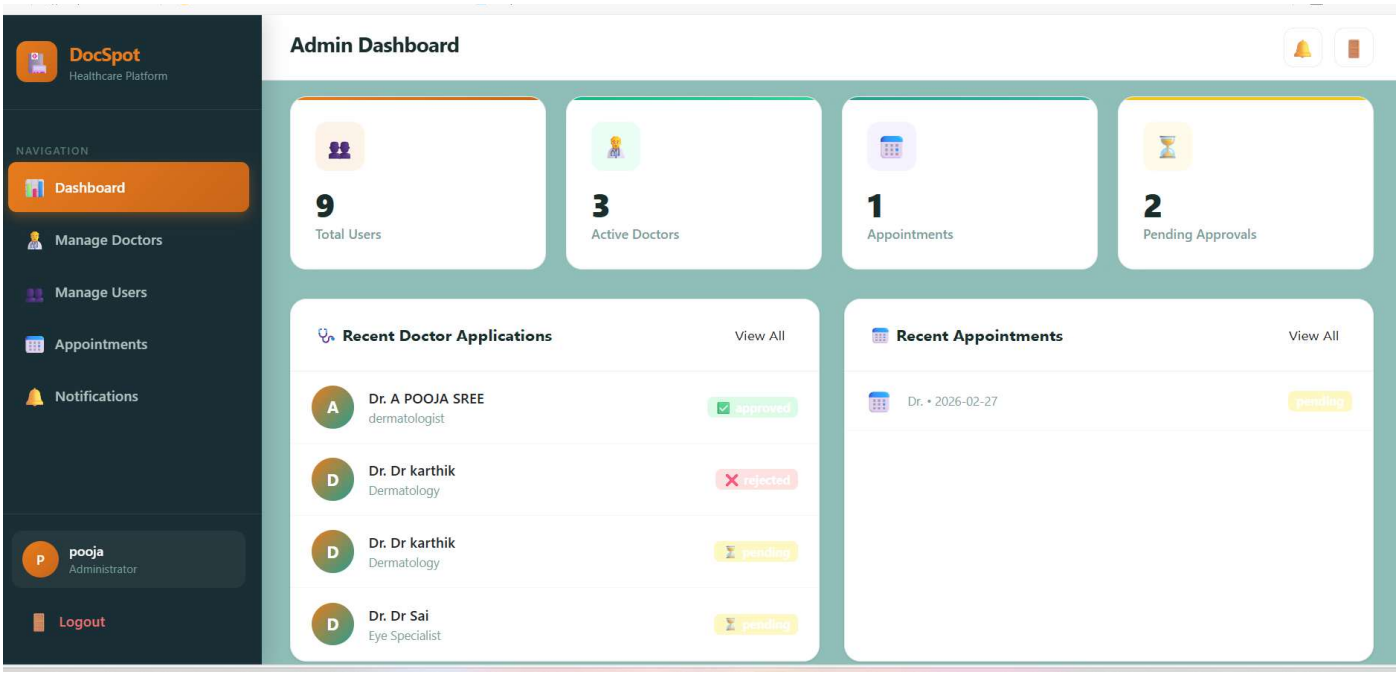


7. RESULTS

User Dashboard:



Admin Dashboard:



7. RESULTS

#	DOCTOR	SPECIALIZATION	EMAIL	PHONE	FEES	EXPERIENCE	STATUS	ACTIONS
1	A Dr. A POOJA SREE	dermatologist		6301415745	₹500	7 yrs	approved	Reject
2	D Dr. Dr karthik	Dermatology	pandupooja1277@gmail.com	+919676839916	₹200	6 yrs	rejected	Approve
3	D Dr. Dr karthik	Dermatology	pandupooja1277@gmail.com	+919676839916	₹200	4 yrs	pending	Approve Reject
4	D Dr. Dr Sai	Eye Specialist	2278a3101@gmail.com	9676839916	₹100	3 yrs	pending	Approve Reject
5	D Dr. Dr jayalakshmi	Orthopedic	pandupooja1277@gmail.com	9121105789	₹250	5 yrs	approved	Reject
6	R Dr. Rudra	General Physician	apoojasree515@gmail.com	9791846835	₹150	7 yrs	approved	Reject

#	USER	EMAIL	PHONE	ROLE	DOCTOR STATUS	ACTIONS
1	P Pooja	karthik23785a3101@gmail.com	6301415735	Admin	—	
2	A A POOJA SREE	akavipoojsindu@gmail.com	akavipoojsindu@gmail.com	User	—	Delete
3	C C.karthik	rebelkarthikpapu@gmail.com	06301415774	User	—	Delete
4	A admin	admin@docspot.com	91900935063	Admin	—	
5	K kavi	jayashalu87@gmail.com	9677312435	User	—	Delete
6	P pooja	kavithaadiyapatham1984@gmail.com	9032590634	Admin	—	
7	S sindhuja	2278a3101@gmail.com	6304028119	User	Doctor	Delete

8. ADVANTAGES &DISADVANTAGES

Advantages:

- Eliminates the need for phone-based or in-person appointment booking.
- Provides a verified doctor directory, increasing patient trust.
- Real-time notifications keep patients informed about appointment status.
- Role-based access ensures data security and appropriate functionality per user.
- Scalable MERN stack architecture supports future feature additions.
- Document upload feature enables doctors to review medical history before appointments.
- Responsive UI ensures accessibility across desktop and mobile devices.

Disadvantages:

- Currently lacks a payment gateway integration for paid consultation booking.
- No real-time chat or video consultation feature between patients and doctors.
- Doctor availability/slots are time-based but not yet integrated with a live calendar system.
- Notification system relies on in-app status updates; no SMS or email notifications yet.
- Admin panel lacks advanced analytics or reporting dashboards.

9. CONCLUSION

DocSpot successfully delivers a comprehensive, role-based appointment booking platform that addressesthecorechallengesfacedbypatients,doctors,andhealthcareadministrators.Byleveraging the MERN stack, the application provides a secure, scalable, and user-friendly solution for managing healthcare appointments digitally.

The system passed UAT with a 96.8% pass rate across 63 test cases, with only two minor open issues related to notifications and date validation. The platform demonstrates strong foundational architecture that can be extended with advanced features in future iterations.

DocSpot represents a meaningful step toward digitizing the healthcare appointment experience, reducing friction for patients, improving efficiency for doctors, and providing administrators with robust oversight tools.

10. FUTURE SCOPE

- **PaymentGatewayIntegration:**IntegrateRazorpayorStripetosupportonlineconsultationfee payments at the time of booking.
- **Real-TimeChat&VideoConsultation:**AddWebSocket-basedchatandWebRTCvideocalls for telemedicine support.
- **Email&SMSNotifications:**IntegrateNodemailerandTwiliotosendappointmentconfirmations and reminders via email and SMS.
- **AI-PoweredDoctorRecommendation:**Implementarecommendationenginethatsuggests doctors based on patient symptoms, history, and ratings.
- **LiveSlotManagement:**Developareal-timecalendar-basedavailabilitysystemfordoctorsto manage time slots dynamically.
- **PatientMedicalRecords:**Createasecure,persistentmedicalhistoryandprescription management module for patients.
- **AnalyticsDashboard:**Buildadmin-facinganalyticswithchartsforappointmenttrends,doctor performance, and user activity.
- **MobileApplication:**DevelopReactNativemobileappsforiOSandAndroidtoextendDocSpot's reach to mobile users.

11. APPENDIX

Source Code

The complete source code for DocSpot is organized into two primary directories:

- **frontend/** — React.js application with pages, components, and API integration.
- **backend/** — Node.js + Express.js server with routes, models, middleware, and config.

Database Collections

- **users** — _id, name, email, password, phone, isDoctor, type, notification
- **doctors**—_id,userID(FK),fullname,email,phone,address,specialization,experience,fees, timings, status
- **appointments** — _id, doctorInfo, userInfo, date, document, status

Key Dependencies

Package	Version	Purpose
express	^4.x	Web framework for Node.js
mongoose	^7.x	MongoDB ODM
jsonwebtoken	^9.x	JWT authentication
bcryptjs	^2.x	Password hashing
multer	^1.x	File upload middleware
cors	^2.x	Cross-Origin Resource Sharing
dotenv	^16.x	Environment variable management
react	^18.x	Frontend UI library
antd	^5.x	UI component library
axios	^1.x	HTTP client for API calls
moment	^2.x	Date/time handling

GitHub & Project Demo Link

GitHub Repository: https://github.com/1112pooja/DocSpot-Seamless-Appointment-Booking-for-Health-.git

Project Demo: https://github.com/1112pooja/DocSpot-Seamless-Appointment-Booking-for-Health-.git